# Reproducing the FilBench leaderboard: harness pitfalls when evaluating Filipino LLMs with lighteval

## PinoAI methods note · METHODS-001

Stan Lee V. Natividad
Kooya PH AI Research
stan@kooya.ph

2026-09-30

> ! Correction (2026-10-05)
>
> A review found the standard-error formula stated inconsistently. The text said SE = √(p(1 − p)/n), but the printed SE and z values (e.g. Belebele 4.99 / −4.8) use lighteval's sample standard error, √(p(1 − p)/(n − 1)). The formula is now stated as the n − 1 form, and the one value computed with the other form, the A100 SIB-200 z, changes from +0.25 to +0.24. No pass/fail verdict changes.

## Abstract

Before comparing candidate base models on FilBench [1], we checked that our evaluation harness reproduces a published leaderboard number. The reference model was Qwen2.5-7B-Instruct, on three log-likelihood multiple-choice (MCF) tasks: Belebele (fil), SIB-200 (tgl) and UniversalNER (ceb).

**The first check failed.** Our harness scored Belebele (fil) at 44.00 against a published 68.11 (z = −4.8). An A/B test traced the whole gap to one setting: lighteval applied the model's chat template to the prompt, so every answer letter was scored as the first token of an assistant turn. With the chat template off, the harness reproduced all three tasks within about one standard error: 65.00 / 79.80 / 97.96 against 68.11 / 77.78 / 97.96. A second platform gave 65.00 / 78.79 / 97.96.

Along the way we found six more pitfalls:

- an upstream lighteval slicing bug that silently breaks token-normalised metrics;
- two GPU memory failures from full-vocabulary logits, the second caused by our own fix;
- detail files lost on Windows because of `|` and `:` in their names;
- datasets that fail to load (gated repositories, and loading scripts refused by `datasets` 4.x);
- output folders that mix files from different runs.

We describe each one's symptom, root cause, fix and verification, and close with a checklist for others evaluating Filipino LLMs. The validation covers MCF tasks only; FilBench's generative tasks are not validated here.

## 1. Introduction

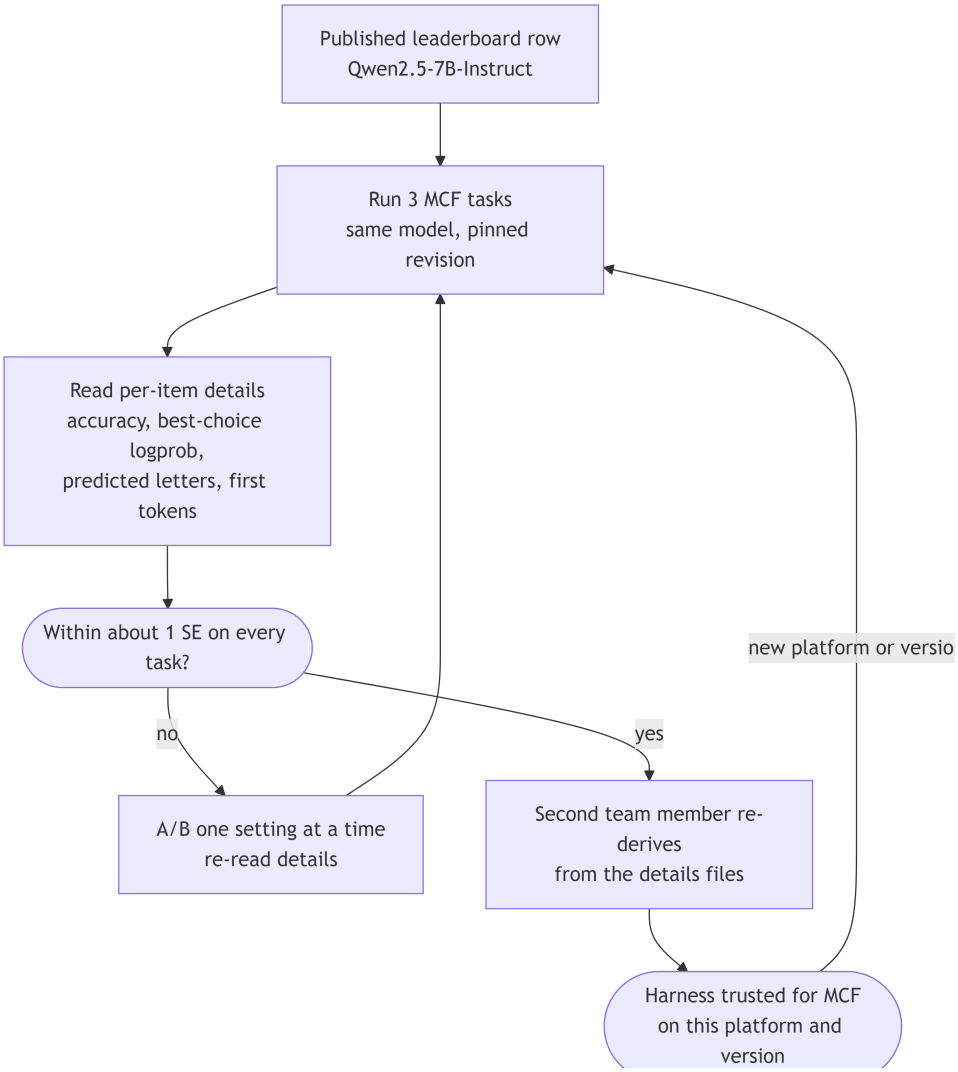
An evaluation harness is itself an experiment. It has settings (prompt format, scoring rule, normalisation, batching) that can move a score by tens of points without any error message. If a harness is used to compare models before it has been checked against a known number, a harness artefact can be mistaken for a model difference.

The cheapest guard is to reproduce a published result first. Pick one model the benchmark authors already scored, run it through your own pipeline on a few tasks, and require agreement within sampling noise before trusting the pipeline for anything else. FilBench [1] publishes per-task scores on a public leaderboard, which makes this possible for Filipino, Tagalog and Cebuano tasks.

We ran this check before a registered base-model comparison. It failed on the first attempt, and the reason was not obvious from the aggregate score. This note records what went wrong and how each problem was found. None of the pitfalls is specific to our models; most apply to anyone running lighteval on FilBench, and several apply to any log-likelihood evaluation.

## 2. Method

- **Reference model:** `Qwen/Qwen2.5-7B-Instruct`, pinned to revision `a09a35458c`, run in fp16.
- **Published reference:** the public FilBench leaderboard, as fetched on 2026-09-29 (space revision `94129c3bec`). For this model it lists Belebele (fil) 68.11, SIB-200 (tgl) 77.78 and UniversalNER (ceb) 97.96, with precision recorded as bfloat16.
- **Tasks:** the lighteval FilBench tasks `belebele_fil_mcf`, `sib200_tgl_mcf` and `universalner_ceb_mcf`, all log-likelihood multiple choice with letter answers.
- **Sample size:** the first 100 items per task (`--max-samples 100`). SIB-200 (tgl) yielded 99 scored items and UniversalNER (ceb) has only 49, so it was evaluated in full.
- **Software:** lighteval 0.13.1.dev0 and transformers 5.17.0, recorded in every run's `run_meta.json` together with the model revision, dtype, batch size and chat-template setting.
- **Platforms:** (1) Kaggle, 2× NVIDIA T4 (about 14.6 GiB each), with the model split across both GPUs; (2) Modal, 1× NVIDIA A100-SXM4-80GB.
- **Pass criterion:** each task must land within about one binomial standard error of the published score, with SE = $\sqrt{(p(1-p)/(n-1))}$ for our accuracy p, i.e. the sample standard error that lighteval reports. At n = 100 and p ≈ 0.65 that is about 4.8 points, so the check detects gross harness errors, not small biases.
- **Diagnostics beyond accuracy:** from each run's per-item details we read (a) the median log-probability of the model's best choice, (b) the distribution of predicted letters against the gold distribution, and (c) the first input tokens, which show the exact prompt format.
- **Verification:** each result was re-derived from the per-item details files by a second team member, independently of the runner's own summary.



# 3. Pitfalls found

## 3.1 The chat template turns letter scoring into noise

**Symptom.** The first run scored Belebele (fil) 44.00 (SE 4.99), against a published 68.11: z = −4.8. SIB-200 (tgl) came in at 72.73 against 77.78 (z = −1.1), and UniversalNER (ceb) at 100.00 against 97.96. The details showed three more warning signs:

- The best choice's log-probability was about −30 nats (median −29.5) for a single answer letter, where a fitting one-token continuation should cost a few nats at most.
- Predictions collapsed onto one letter: B 72 times out of 100, while B was correct only 33 times.
- UniversalNER (ceb) was unaffected: its prediction counts matched gold exactly. That task is easy enough that the signal survives even a badly scored prompt.

**Root cause.** lighteval applies the tokenizer's chat template whenever one exists, unless told otherwise. Every Belebele input therefore began with Qwen's `<|im_start|>` token and default system prompt, and each answer letter was scored as the first token of the assistant's reply. The model finds a bare letter very unlikely in that position, so the four letter log-probabilities were all tiny and nearly equal, and the argmax was driven by noise.

**A/B test.** We changed one setting per arm on Belebele (fil), n = 100:

- **Arm (i):** chat template off, memory patch on (see 3.3). Accuracy 65.00, median best-choice log-probability −0.068, predictions 22/30/23/25 across A/B/C/D against a gold spread of 21/33/25/21. The input now starts at the passage text, with no `<|im_start|>`.
- **Arm (ii):** chat template on, memory patch off. Accuracy 44.00, median −29.5, predictions 15/72/3/10. That is identical to the failing run, digit for digit.

Arm (ii) also cleared our own memory patch as a suspect, since removing it changed nothing.

**Fix.** Chat template off for every FilBench log-likelihood MCF run, for base and instruct models alike. The runner now defaults to this whenever all selected tasks are MCF, and records the setting in `run_meta.json`.

**Verification.** The three-task check was repeated with the template off: 65.00 / 79.80 / 97.96 against 68.11 / 77.78 / 97.96. The second reviewer confirmed from the details that no input starts with `<|im_start|>` and that median best-choice log-probabilities were −0.068, −0.028 and −0.0001.

## 3.2 An upstream slicing bug breaks token-normalised metrics

**Symptom.** A later run on KALAHI crashed with an `IndexError` inside lighteval's token-length normalisation (`LogProbTokenNorm`). Belebele, SIB-200 and UniversalNER had never crashed.

**Root cause.** On the multi-GPU (accelerate gather) path, lighteval's `transformers_model.py` slices the gathered continuations as `[:num_choices][:len_choice]`. That slices the **rows** twice, so each item keeps only `len_choice` of its choices instead of all of them. Our own details files showed it: the stored output tokens had length 1 (template off) or 2 (template on) per item, against 4 choices, with and without our memory patch. A small numpy reproduction of the slice gave the same result. Plain accuracy does not use the per-choice token lengths, so it is unaffected. Metrics that divide by choice length (KALAHI, and likely Global-MMLU and StingrayBench) fail or would be wrong. The bug was present at the lighteval commit pinned in our project and on lighteval's main branch when it was found.

**Fix.** The runner patches the installed lighteval file right after installation, replacing the slice with `[:num_choices, :len_choice]` (rows, then columns). It exits if the pattern is not found exactly once, and logs `lighteval gather-slice patch: applied`.

**Verification.** The KALAHI retest on a 4B base model completed: the output tokens matched the number of choices on every row, and all three metrics (acc, acc_norm_token, acc_norm) were present. The validated accuracies in 3.1 are unchanged, because accuracy never depended on the sliced field.

## 3.3 Full-vocabulary logits exhaust a T4

**Symptom.** Runs on 2× T4 ran out of memory on GPU 0 at the first log-probability batch, for example "Tried to allocate 1.57 GiB … 13.06 GiB allocated by PyTorch" out of 14.56 GiB. The first failure was already at batch size 1, and capping GPU 0's share of the weights did not fix it.

**Root cause.** lighteval computes logits for every position of the sequence, over the whole vocabulary, although MCF scoring only needs the few positions of each answer continuation. With the model split across GPUs, the output lands

on GPU 0, on top of that GPU's share of the weights. Even at batch size 1, all of an item's choices go through one forward pass. Our estimate, not a measurement: for a ~1,000-token prompt with 7 choices, the fp16 logits plus an fp32 log-softmax copy take about 5.9 GiB with a 151,936-token vocabulary, and about 10 GiB with the 248K–262K vocabularies of newer models.

**Fix.** A wrapper passes `logits_to_keep` to the Hugging Face model, keeping only the last positions of the batch plus a margin of 32. It returns an object that lighteval can index by the original absolute positions, so lighteval's scoring code is untouched, and it raises an error if a needed position was not kept.

**Verification.** On CPU, a test mirroring lighteval's scoring loop gave identical per-choice log-probability sums and exact-match flags with and without the wrapper. On GPU, arm (ii) of the A/B (3.1) turned the wrapper off and reproduced the failing run exactly, so the wrapper does not change scores.

### 3.4 The memory fix fails on mixed-length batches

**Symptom.** Full-sample runs of three base models at batch size 4 ran out of memory again at the first log-probability batch (for one model, an attempted allocation of 10.09 GiB), although short two-task smoke runs had passed.

**Root cause.** The wrapper in 3.3 keeps **one** contiguous window, sized from the shortest real row in the batch. With right padding, one short item in a batch of long ones (for example, short items next to long readability passages) makes the window almost the whole sequence, which brings back the full-sequence logits. The smoke runs passed because their tasks have similar lengths within a batch. The 10.09 GiB allocation fits this: at 2 bytes per logit over a 248K vocabulary, it is about 21K kept row-positions.

**Fix.** Either run MCF at batch size 1, so each forward pass holds one item's choices of similar length, or move to a GPU where full logits fit. We moved to a single A100 80 GB with the wrapper off. A per-row index set (the union of each row's last positions, which `logits_to_keep` also accepts) would fix the wrapper itself; we have not built it.

**Verification.** The platform change was treated as a harness change, so the three-task check was rerun on the new platform before any model comparison (Table 1).

### 3.5 Windows loses detail files silently

**Symptom.** Downloading run outputs to a Windows machine failed in two ways:

- With `|` in a name, the download aborted with `[Errno 22]` before the results summary and run metadata arrived, although the remote run had completed.
- With `:` in a name, the file arrived as a single empty (0-byte) file, and nothing reported an error. For Global-MMLU, the per-item details of all 57 subjects were lost this way, while the results summary still showed an aggregate score.

**Root cause.** lighteval names its details files `details_<task>|0_<timestamp>.parquet`, and Global-MMLU tasks as `details_global_mmlu_all_tgl_mcf:<subject>|0_<timestamp>.parquet`. Windows cannot create `|`. On NTFS, `name:stream` is the syntax for an alternate data stream, so everything after the `:` went into a hidden stream of one empty file.

**Fix.** Rename inside the remote container, before anything is downloaded: `|` becomes `__` and `:` becomes `--`. The local download step applies the same mapping again as a second guard. The scorer refuses any run with a 0-byte details file or zero Global-MMLU items, so a lossy fetch cannot shrink a comparison unnoticed.

**Verification.** After the fix, each base model's full run arrived with 69 details files, 57 of them for Global-MMLU, and no 0-byte files.

### 3.6 Dataset loading: one bad task kills the whole run

**Symptom.** Two early runs died within four minutes, before any scoring: one with `DatasetNotFoundError` on a gated Tatoeba dataset, the other with "Dataset scripts are no longer supported, but found StingrayBench.py".

**Root cause.** lighteval loads every task's dataset up front, so a single failing dataset ends the run. Four FilBench tasks sit behind gated Hugging Face repositories, which need an access token. `datasets` 4.x no longer runs repository loading scripts, and the StingrayBench and TICO-19 repositories ship one.

**Fix.** The runner skips gated tasks unless explicitly asked to include them (with a token available), and skips script-loader tasks by default. These can be restored by loading them from the repository's auto-converted parquet branch instead of the script.

**Verification.** Later runs on the default task set loaded cleanly. A run including the gated tasks with a token completed, and a StingrayBench run loaded from parquet completed as a plumbing check.

### 3.7 Output folders mix runs

**Symptom and root cause.** Three separate provenance problems appeared, none of which changes a score but each of which could attach one run's numbers to another:

- After an A/B arm, the whole output folder was copied, and lighteval's details persist across runs in that folder. The arm (ii) folder therefore also held arm (i)'s files, and a naive file glob picked the wrong arm.
- On Kaggle, a refused kernel push was printed but not treated as a failure. The runner then read the previous version's "complete" status and downloaded that version's output under the new model's name.
- On Modal, a warm container was reused between calls, and its output folder still held the previous model's files.

**Fix.** Copy only files named by the run's own timestamp. Treat a push error as a failure. Empty the output folder at the start of every call. After download, check that `run_meta.json` names the requested model and revision. Misplaced files were moved aside, not deleted.

**Verification.** Every result in Table 1 was matched to its run identifier and re-read from its own details files by a second team member.

## 4. Results

Table 1: Qwen2.5-7B-Instruct, fp16, accuracy × 100 on the first 100 items per task, against the published FilBench leaderboard. z = (ours − published) / SE, with SE = √(p(1 − p)/(n − 1)), the sample standard error lighteval reports for our accuracy. "—" means the task was not part of that run.

| Run | Belebele (fil), n = 100 | SIB-200 (tgl), n = 99 | UniversalNER (ceb), n = 49 | Verdict |
|---|---|---|---|---|
| Published leaderboard (bfloat16) | 68.11 | 77.78 | 97.96 | — |
| T4, chat template on (first check) | 44.00 (z = −4.8) | 72.73 (z = −1.1) | 100.00 | fail |
| T4, chat template on, memory patch off | 44.00 | — | — | fail |
| T4, chat template off (A/B arm i) | 65.00 | — | — | pass |
| T4, chat template off (full recheck) | 65.00 (z = −0.65) | 79.80 (z = +0.5) | 97.96 (48/49) | pass |
| A100, chat template off, memory patch off | 65.00 (z = −0.65) | 78.79 (z = +0.24) | 97.96 | pass |

With the chat template off, the harness reproduces the published numbers within about one standard error on both platforms. On the two platforms, Belebele and UniversalNER agree item for item, and SIB-200 differs by one item. The median best-choice log-probability on Belebele moved from −29.5 with the template on to −0.068 with it off. The first check's failure was entirely the prompt format: removing our memory patch reproduced it exactly, and removing the chat template fixed it.

## 5. Recommendations for evaluating Filipino LLMs

- ☐ **Reproduce a published number first.** Use one model the benchmark authors scored, on two or three tasks, and set the pass criterion (for example, within about 1 SE) before running.
- ☐ **Decide the chat template explicitly for log-likelihood tasks.** For FilBench MCF, turn it off for every model, base and instruct. Record the setting with every result.
- ☐ **Look past accuracy.** Check the best-choice log-probabilities (a few nats or less for a one-token letter), the predicted-letter distribution against gold, and the first input tokens of a few items.
- ☐ **Do not trust an easy task as a canary.** A near-ceiling task (here UniversalNER ceb) can pass under a broken prompt format.
- ☐ **Patch or check lighteval's multi-GPU gather slice** before using token-normalised metrics (KALAHI, and likely Global-MMLU and StingrayBench). Confirm that stored output tokens match the number of choices.
- ☐ **Budget memory for full-vocabulary logits** on small GPUs, especially with vocabularies above 150K. If you keep only the last logits, size the window per row, or use batch size 1.
- ☐ **Rename `|` and `:` in lighteval file names before they reach Windows**, and refuse runs with 0-byte details files.
- ☐ **Expect dataset loading to fail as a whole.** Exclude gated and script-loader tasks explicitly, or load script-based datasets from their parquet conversion.
- ☐ **Treat every platform, dtype or library change as a harness change** and rerun the reference check.

- ☐ **Keep provenance per run:** a run identifier in `run_meta.json`, output folders cleared per run, and details re-read by a second person.

## 6. Limitations

- **Small samples.** n = 100, 99 and 49. One standard error is about 4–5 points on Belebele and SIB-200, so this check rules out gross harness errors but cannot detect a bias of a point or two. Our Belebele score sits 3.1 points below the published value on both platforms, which is within noise but not evidence of zero bias.
- **Subset against full set.** Our scores cover the first 100 items of each task, while the published values are computed over each task's full evaluation data. UniversalNER (ceb) is the exception, as we ran all 49 items.
- **MCF only.** Only log-likelihood multiple-choice tasks were validated. FilBench's generative tasks (translation and others), and the chat-template setting they need, are **not** validated by this check and need their own reference comparison.
- **Precision.** We ran fp16; the leaderboard lists bfloat16 for this model.
- **Library version.** The validation holds for the lighteval build we ran (0.13.1.dev0, installed from the main branch, plus the gather-slice patch) and transformers 5.17.0. A different lighteval version needs the check rerun.
- **One reference model.** A single instruct model was used. The chat-template rule was adopted for base models by reasoning rather than by a separate base-model reproduction.
- **Estimates.** The memory figures in 3.3 are arithmetic estimates checked against observed allocations, not profiler measurements.

## Acknowledgments

We thank the leadership of Kooya PH for supporting this work, and the native-speaker evaluators who reviewed and rated the Filipino and Taglish material.

**AI assistance.** Analysis code, statistical checks and drafts of this document were prepared with the help of Claude (Anthropic), an AI model. Every number is generated from the project's result files. The author reviewed the content and takes responsibility for it.

## References

Benchmark: L. J. V. Miranda and others [1].

[1] L. J. V. Miranda and others, "FilBench: Can LLMs Understand and Generate Filipino?," *arXiv preprint arXiv:2508.03523*, 2025.